

MIPS Processor Comparison

16-Bit Single-Cycle vs 32-Bit Single Cycle
Implemented in Verilog



Sean Wygant

Heba Hafez

Daniel Mondragon

Ricardo Herrera-Pineda

ECE 4300: Computer Architecture

November 2025

Project Overview

Project Mission

Design, implement, and benchmark two MIPS processors using Verilog HDL: a 16-bit single-cycle processor and a 32 single-cycle processor to analyze performance differences in architectural design.

Build

Two MIPS processors

Test

Run test programs

Compare

Measure performance

Research Questions

RQ1: Does data path width directly correlate with hardware resource utilization?

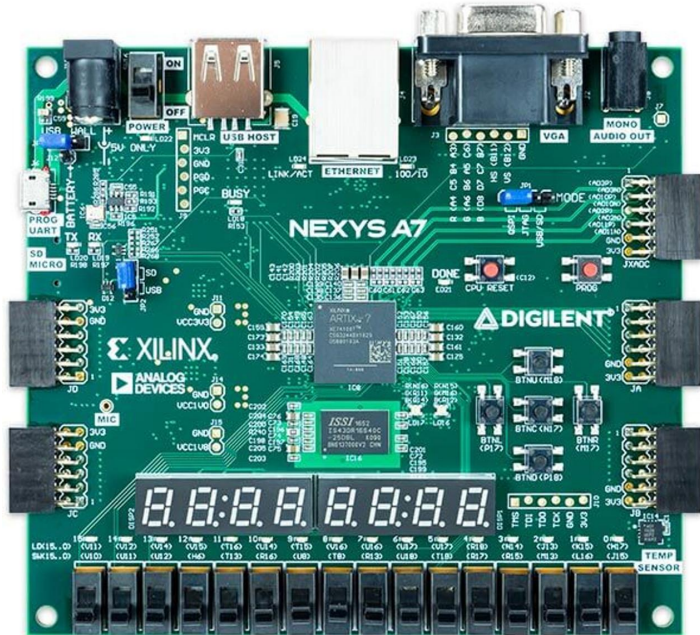
**RQ2: How do algorithmic design choices
impact resource consumption independent of data width?**

RQ3: What are the performance implications of different branch calculation strategies?

RQ4: Can these findings inform future educational processor implementations?

Tools we used:

Nexys A7 100T



AMD Xilinx Vivado



Tools & Environment

Hardware

- Nexys A7-100T FPGA
- Artix-7 XC7A100T
- 63,400 LUTs
- 126,800 Flip-Flops

Software

- Xilinx Vivado 2023.2
- Verilog HDL
- Synthesis & Implementation
- Power Analysis Tools

Our Approach: Discovery Method

Phase 1: DEPLOYMENT

Deployed both processors on identical FPGA hardware

Phase 2: MEASUREMENT

Collected comprehensive metrics (LUTs, power, timing, performance)

Phase 3: DISCOVERY

Found unexpected paradox: 16-bit uses MORE resources!

Phase 4: INVESTIGATION

Analyzed code to find root cause (branch logic differences)

Phase 5: CHARACTERIZATION

Quantified impact across all metrics

What is MIPS?

MIPS stands for Microprocessor without Interlocked Pipeline Stages. It's a RISC architecture known for its simplicity and educational value.

Single-Cycle Architecture

1

Each instruction completes in exactly one clock cycle

2

Simpler to understand than pipelined designs

3

Clock period set by longest instruction path

Key Architectural Differences

Execution Model

Single-Cycle: One instruction per cycle

Pipelined: Multiple instructions in flight

Data Path Width

16-bit: 16-bit buses and operations

32-bit: 32-bit buses and operations

Clock Cycle Time

Single-Cycle: Slowest instruction sets clock

Pipelined: Shortest stage sets clock

Complexity

Single-Cycle: Simple, no hazards

Pipelined: Needs forwarding, stalls, flushes

Performance Trade-offs

Pipelined architecture offers higher throughput but adds complexity. 32-bit handles larger data but requires more hardware resources. We chose to use 32-bit and 16-bit single cycle variants due to the simplicity and speed of implementation.

Architectural Differences

16-Bit Single-Cycle

- Each instruction in one cycle
- 16-bit data path
- 8 registers (3-bit addressing)
- 13 instructions
- Max memory space: 64 KB

32-Bit Single-Cycle

- Each instruction in one cycle
- 32-bit data path
- 32 registers (5-bit addressing)
- 13 instructions (we can go up to ~8192)
- Max memory space: 4 GB

MIPS Instruction Formats

All Instructions are 16 or 32 bits depending on architecture

R-type

Register operations

add, sub, and, or, slt, jr

I-type

Immediate operations

addi, lw, sw, beq, slti

J-type

Jump instructions

j, jal

Name	Format	Example					Comments
		3 bits	3 bits	3 bits	3 bits	4 bits	
add	R	0	2	3	1	0	add \$1,\$2,\$3
sub	R	0	2	3	1	1	sub \$1,\$2,\$3
and	R	0	2	3	1	2	and \$1,\$2,\$3
or	R	0	2	3	1	3	or \$1,\$2,\$3
slt	R	0	2	3	1	4	slt \$1,\$2,\$3
jr	R	0	7	8	0	8	jr \$7
lw	I	4	2	7	7		lw \$1, 7 (\$2)
sw	I	5	2	7	7		sw \$1, 7 (\$2)
beq	I	6	1	7	7		beq \$1,\$2,7
addi	J	7	2	1			addi \$1,\$2,7
j	J	2	500				j 1000
jal	J	3	500				jal 1000
slti	I	1	2	1	7		slti \$1,\$2, 7

Processor Datapath

Key Components

Program Counter

Tracks current instruction

Instruction Memory

Stores program code

Register File

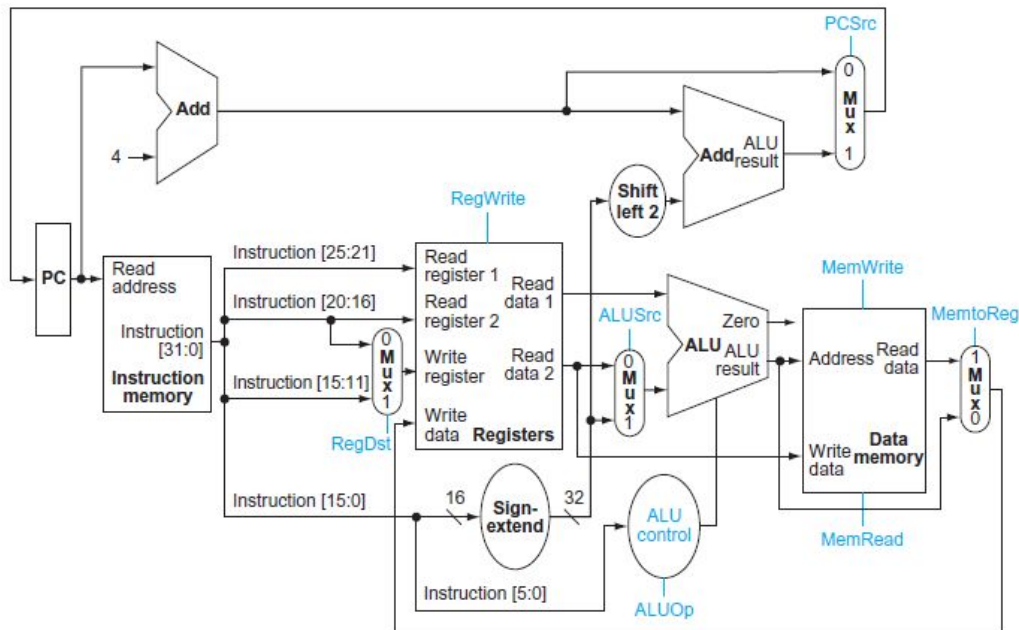
8 or 32 general-purpose registers

ALU

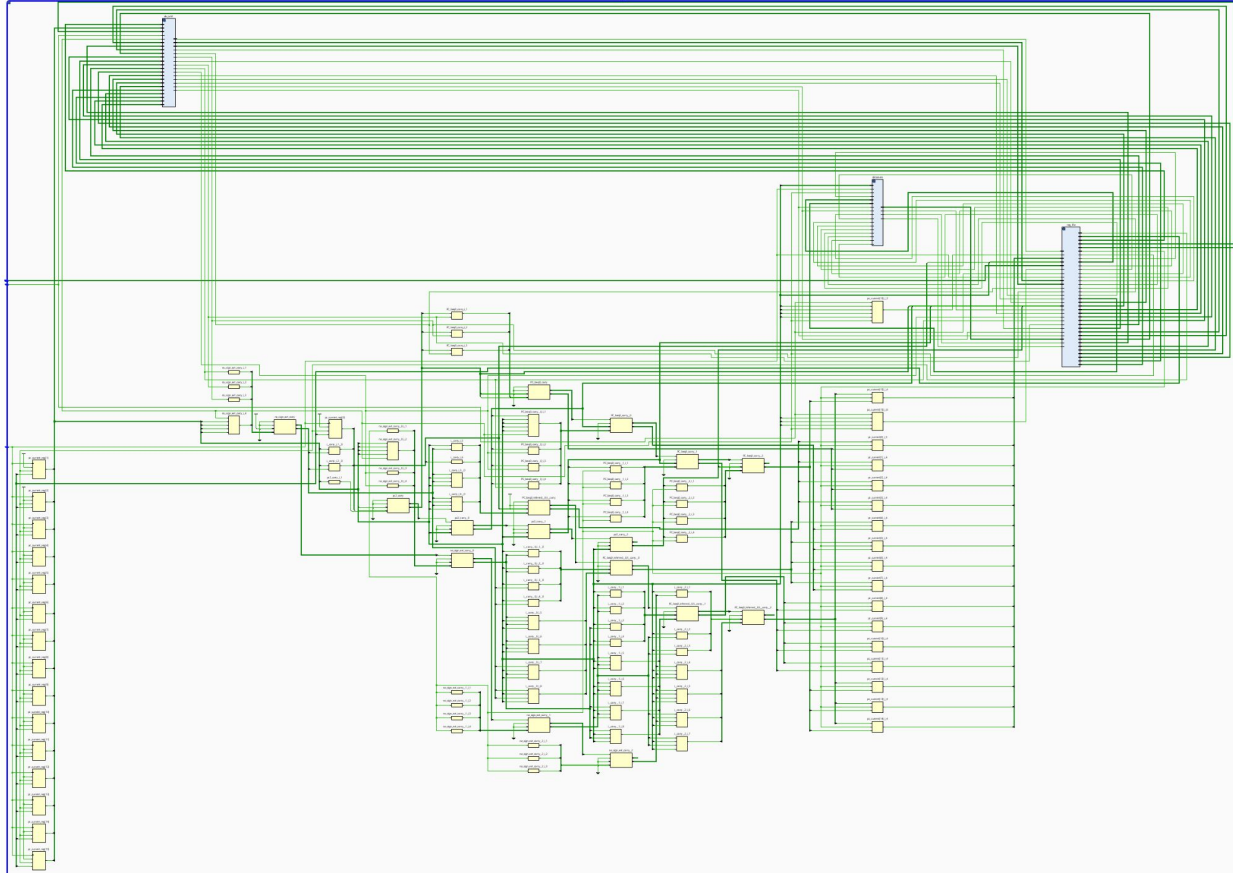
Performs arithmetic operations

Data Memory

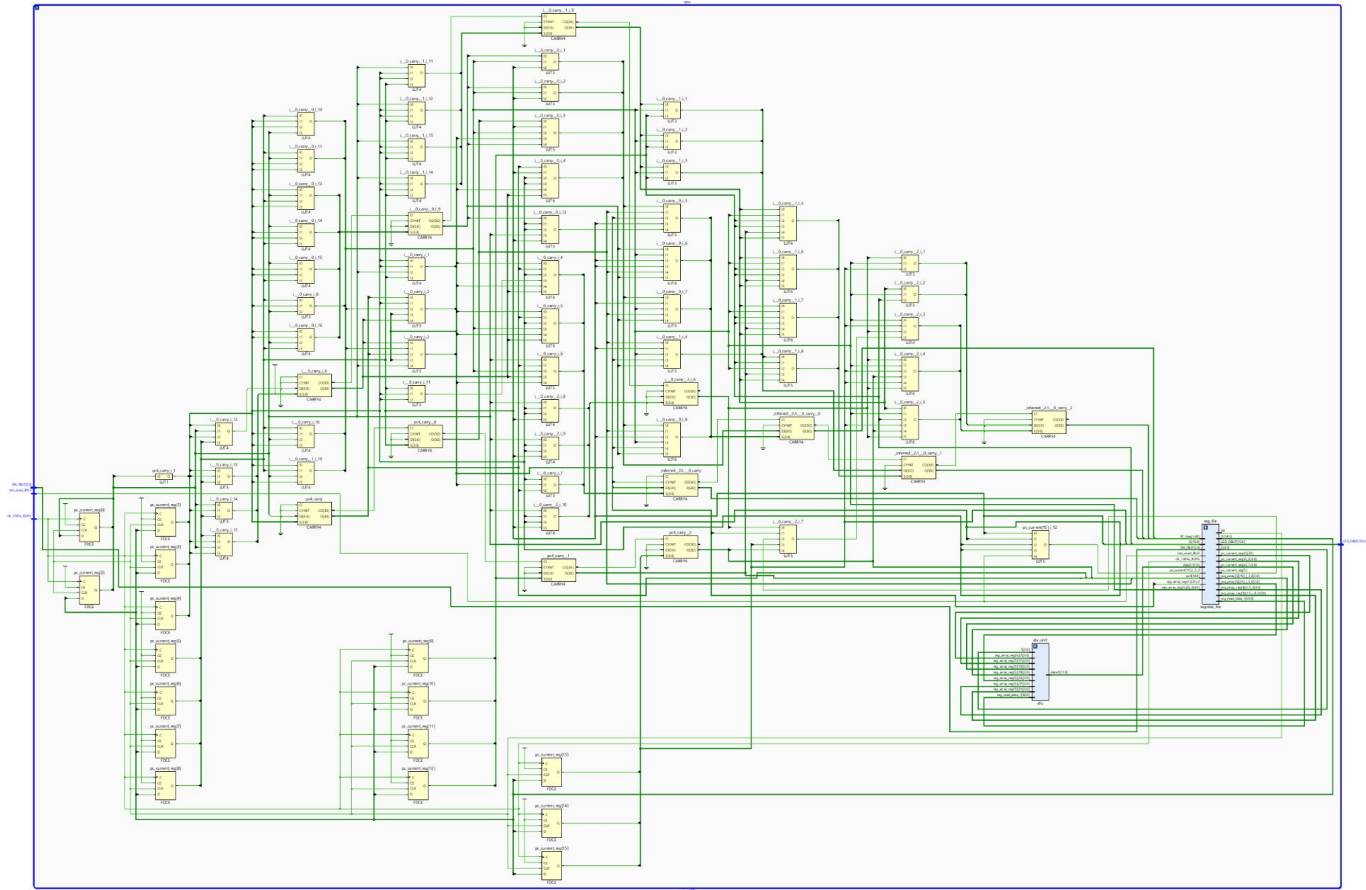
Load and store data



16 Bit Single-Cycle Processor Schematic in Verilog



32 Bit Single-Cycle Processor Schematic in Verilog



The instructions for both architectures

16-Bit

```
1 `timescale 1ns / 1ps
2 // Verilog code for instruction memory
3 module instr_mem // a synthesizable rom implementation
4 (
5     input [15:0] pc,
6     output wire [15:0] instruction
7 );
8     wire [3 : 0] rom_addr = pc[4 : 1];
9     /*
10     Nested for loop
11     for (i=0, i<64, ++i) begin
12         k = 0;
13         for (j=0, j<64, ++j) begin
14             k = k + j;
15         end
16         LED = i;
17     end
18     */
19     reg [15:0] rom[15:0];
20     initial
21     begin
22         rom[0] = 16'b1000001010000000; //LW $5 0($0) Set Register 5 to 0
23         rom[1] = 16'b0011010100111111; //SLTI $2 $5 10 Set register 2 to 1 if register 5 is less than 63
24         rom[2] = 16'b1000000110000000; //LW $3 0($0) Set Register 3 to 0
25         rom[3] = 16'b1000001000000000; //LW $4 0($0) Set Register 4 to 0
26         rom[4] = 16'b1100100000001000; //BEQ $2 $0 (branch to ROM13)
27         rom[5] = 16'b0010110010111111; //SLTI $1 $3 50Set register 1 to 1 if register 3 is less than 63
28         rom[6] = 16'b1000100000000001; //BEQ $1 $0 (branch to ROM10)
29         rom[7] = 16'b0001000111000000; //ADD $4 $4 $3 (Register 4 counts to 2016)
30         rom[8] = 16'b1110110110000001; //ADDI $3 $3 1 (add 1 to register 3 and store in register 3)
31         rom[9] = 16'b1100000001111011; //BEQ $0 $0 (branch to ROM5)
32         rom[10] = 16'b1110110100000001; //ADDI $5 $5 1 (add 1 to register 5 and store in register 5)
33         rom[11] = 16'b110000000110101; //BEQ $0 $0 (branch to ROM1)
34         rom[12] = 16'b0000000000000000;
35         rom[13] = 16'b0000000000000000;
36         rom[14] = 16'b0000000000000000;
37         rom[15] = 16'b0000000000000000;
38     end
39     assign instruction = {pc[15:0] < 32 }? rom[rom_addr[3:0]] : 16'd0;
40 endmodule
```

32-Bit

```
1 `timescale 1ns / 1ps
2 module instr_mem(
3     input [31:0] pc,
4     output [31:0] instruction
5 );
6     wire [3:0] rom_addr = pc[5:2];
7     integer i;
8     reg [31:0] rom [0:31];
9
10    /* for (i=0; i<64; ++i) {
11        k = 0;
12        for (j=0; j<64; ++j) {
13            k += j;
14        }
15        LED = k;
16    }
17    */
18
19    initial begin
20        for (i = 0; i < 32; i = i + 1) rom[i] = 32'b0;
21
22        rom[0] = 32'b000111_00000_00101_0000000000000000; // addi $5,$0,0
23        rom[1] = 32'b000111_00000_00001_0000000000100000; // addi $1,$0,64
24        rom[2] = 32'b000110_00101_00001_0000000000001010; // beq $5,$1,+10
25        rom[3] = 32'b000111_00000_00011_0000000000000000; // addi $3,$0,0
26        rom[4] = 32'b000111_00000_00100_0000000000000000; // addi $4,$0,0
27        rom[5] = 32'b000110_00011_00001_0000000000000100; // beq $3,$1,+4
28        rom[6] = 32'b000000_00100_00011_00100_000000000000; // add $4,$4,$3
29        rom[7] = 32'b000111_00011_00011_0000000000000001; // addi $3,$3,1
30        rom[8] = 32'b000110_00000_00000_1111111111111100; // beq $0,$0,-4
31        rom[9] = 32'b00000000000000000000000000000000;
32        rom[10] = 32'b000000_00101_00000_00100_000000000000; // add $4,$5,$0
33        rom[11] = 32'b000111_00101_00101_0000000000000001; // addi $5,$5,1
34        rom[12] = 32'b000110_00000_00000_1111111111110100; // beq $0,$0,-12
35        rom[13] = 32'b000110_00000_00000_1111111111111111; // beq $0,$0,-1
36    end
37
38    assign instruction = rom[rom_addr];
39 endmodule
```

Test Program

Summation Algorithm

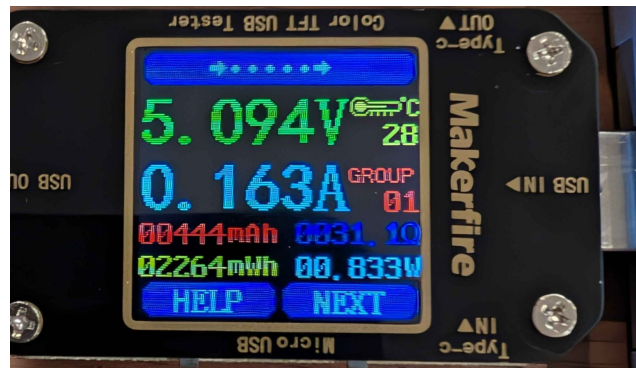
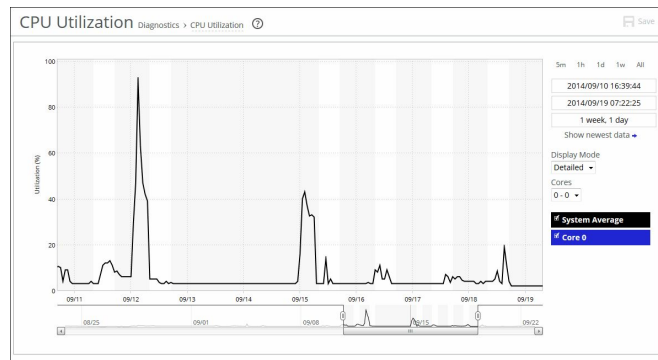
$$\text{Sum} = 0 + 1 + 2 + 3 + \dots + 48 + 49$$

- Register \$3: Tracks current value of n
- Register \$4: Stores running sum (displayed on LEDs)
- Loop limit: 49 (hardcoded)
- Expected result: 1,225 (using formula $n(n+1)/2$)

Performance Metrics

Metrics to Measure

- Execution time
- Clock cycles per instruction (CPI)
- Power consumption
- CPU utilization
- Temperature



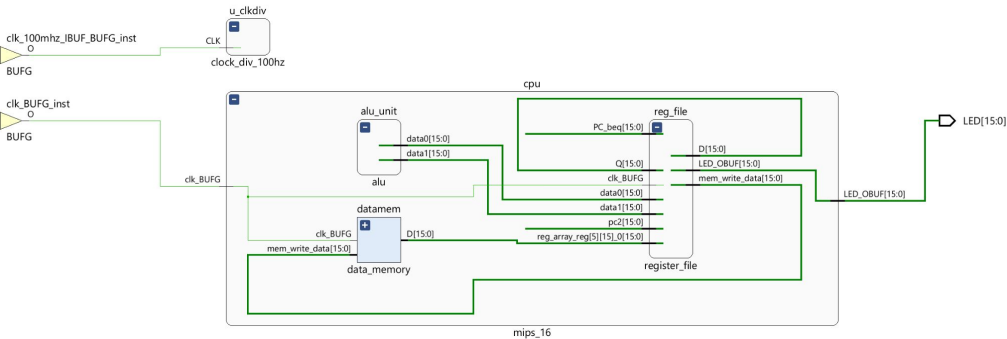
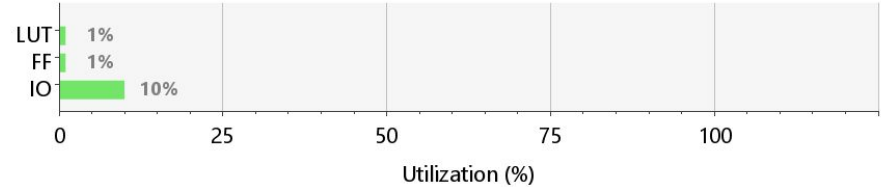
Results

- *MIPS 16 executes 20,354 instructions*
- *MIPS 32 executes 16,836 instructions*
- **MIPS 16 requires 21% more instructions** to perform the same calculation.
- *MIPS 16 executes the summation loop in 8.07s*
- *MIPS 32 executes the summation loop in 6.70s*
- **Overall speedup of 21%** due to the decrease in instruction count.
- *MIPS 16 requires 0.833W using a 10 khz clock*
- *MIPS 32 requires 0.862W using a 10 khz clock*
- **MIPS 32 uses just 4% more power** to perform the same calculation.

Board Utilization Tables

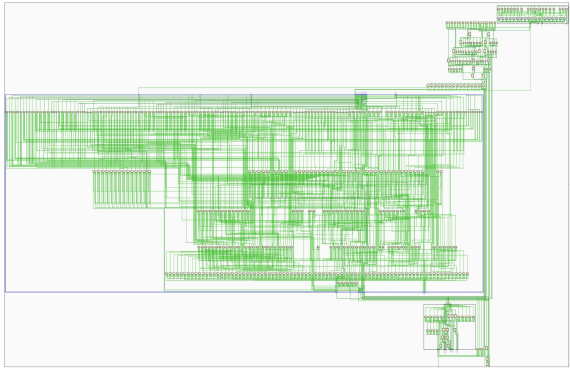
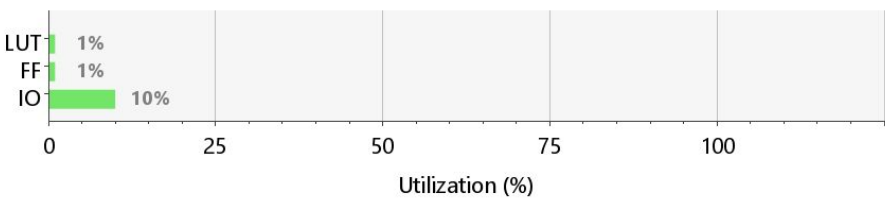
16-Bit

Resource	Utilization	Available	Utilization %
LUT	414	63400	0.65
FF	116	126800	0.09
IO	21	210	10.00



32-Bit

Resource	Utilization	Available	Utilization %
LUT	376	63400	0.59
FF	196	126800	0.15
IO	21	210	10.00



Resource Utilization Analysis

Resource	16-Bit	32-Bit	Winner
LUTs	414 (0.65%)	376 (0.59%)	32-bit ✓
Flip-Flops	116 (0.09%)	196 (0.15%)	16-bit ✓
I/O	21 (10.0%)	21 (10.0%)	Tie

Key Insight: Algorithmic Efficiency Matters!

- 16-bit uses MORE LUTs: Inefficient branch logic (5 operations vs 2)
- 32-bit uses MORE FFs: Wider data paths need more storage (expected!)
- The lesson: Smart algorithms beat brute-force narrowing

Root Cause: Branch Calculation

Why does 16-bit use MORE LUTs?

16-Bit Branch Logic

1. Shift immediate left
2. Negate (NOT + add 1)
3. Check if negative
4. Subtract OR Add
5. Select result (MUX)

Total: 5 operations

32-Bit Branch Logic

1. Shift immediate left
2. Add to PC

Total: 2 operations

Power Draw

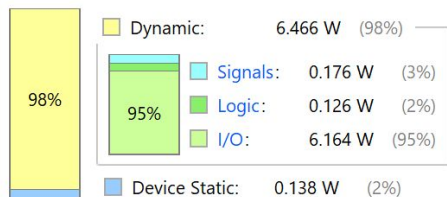
16-Bit

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	6.604 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	55.1°C
Thermal Margin:	29.9°C (6.5 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	4.6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



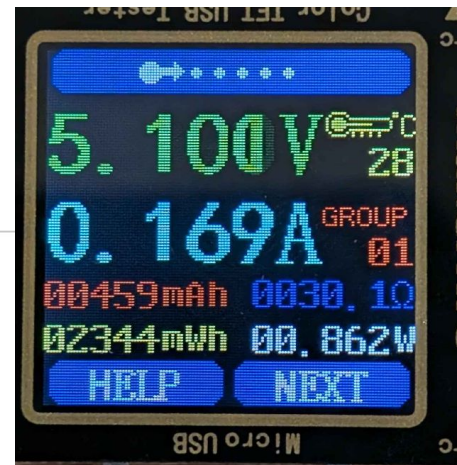
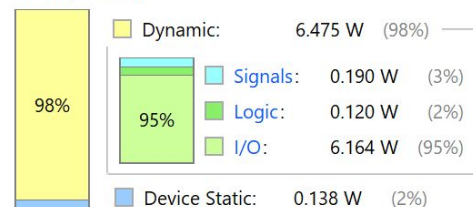
32-Bit

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	6.613 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	55.2°C
Thermal Margin:	29.8°C (6.5 W)
Ambient Temperature:	25.0 °C
Effective θ_{JA} :	4.6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



Power Draw Analysis

16-Bit

Total Power: 6.604 W

Dynamic: 6.466 W (98%)

Signals: 0.176 W (3%)

Logic: 0.126 W (2%)

I/O: 6.164 W (95%)

32-Bit

Total Power: 6.613 W

Dynamic: 6.475 W (98%)

Signals: 0.190 W (3%)

Logic: 0.120 W (2%)

I/O: 6.164 W (95%)

I/O dominates (95%)! 32-bit uses +8% signal power, -5% logic power → Nearly equal total

Power Analysis: Power vs Energy

Instantaneous Power (W)

16-bit: 6.604 W ✓

32-bit: 6.613 W ✗

32-bit uses +0.009W MORE

Why? Wider signal buses

Signal power: 0.190W vs 0.176W

Total Energy (J)

16-bit: 6.72 J ✗

32-bit: 5.78 J ✓

32-bit uses 14% LESS energy

Why? Finishes 21% faster

Speed advantage > power cost

Like a car: Burns more fuel/hour but uses less total fuel because it arrives sooner!

Timing Reports (Clock was set to 10kHz)

16-Bit

Timing Check	Count  1	Worst Severity
unconstrained_internal_endpoints	887	 High
no_clock	180	 High
no_output_delay	16	 High
no_input_delay	1	 High
constant_clock	0	
pulse_width_clock	0	
multiple_clock	0	
generated_clocks	0	
loops	0	
partial_input_delay	0	
partial_output_delay	0	
latch_loops	0	

32-Bit

Timing Check	Count  1	Worst Severity
unconstrained_internal_endpoints	551	 High
no_clock	196	 High
no_output_delay	16	 High
no_input_delay	1	 High
constant_clock	0	
pulse_width_clock	0	
multiple_clock	0	
generated_clocks	0	
loops	0	
partial_input_delay	0	
partial_output_delay	0	
latch_loops	0	

Timing Reports (Clock: 10 kHz)

16-Bit

32-Bit

unconstrained_internal_endpoints

887

551

no_clock

180

196

16-bit has 61% MORE endpoints (887 vs 551)

Complex branch logic creates more intermediate signals and registered stages

At 10 kHz, both have enormous timing margin—no actual violations!

Videos

Mips16 10kHz execution time @ 0.25x, speed = 8.07s



Mips32 10kHz execution time @ 0.25x, speed = 6.70s



Key Findings Summary

Performance Winner: 32-bit ✓

- **Instruction Efficiency:**

21% fewer instructions, 21% faster execution

- **Resource Efficiency:**

9% fewer LUTs (better algorithm!), more FFs (expected)

- **Power Efficiency:**

Nearly equal total power (~4% more), trades logic for signal power

Conclusions

What We Learned

- Architecture matters more than width alone
- 16-bit used MORE LUTs despite being narrower
- **Root cause: Inefficient branch calculation algorithm**
- **Design principles observed:**
 - Simple algorithms → Fewer LUTs → Less logic power
 - Wider data paths → More FFs → More signal power
 - Good design balances complexity and width

Key Takeaway: Smart algorithms can overcome architectural limitations!